Coding Club Resource Hub- Sprint 0 Meeting Summary

Date: May 6, 2026
Speaker: Hakim Rashid

1. Introduction & Project Vision

The meeting commenced with a land acknowledgement recognizing the traditional, ancestral, and unceded territory of the Syilx Okanagan Nation. The core motivation behind the CS Community Platform is to address the lack of centralized resources and guidance for first-year and incoming Computer Science, Math, Physics, and Statistics (CMPS) students.

1.1. Core Objectives

- Create a centralized repository for the CMPS department.
- Provide accessible resources (worksheets, past papers, advice) to help students navigate their academic pathways.
- Bridge the information gap regarding research opportunities, internships, and campus clubs.
- Foster a collaborative environment built and maintained by the student community.

2. Platform Features & Architecture

The application is divided into two primary structural components to serve both static information and dynamic community interaction.

2.1. Static Pages (Information Hub)

- **Welcome & About Us:** General overview and team introductions.
- **Clubs & Events:** Information on campus clubs (club based experience) and an archive of past events.
- **Advice:** Guides on navigating different academic and career pathways (e.g., internships, research).
- **Professor Research Pages:** Dedicated profiles for professors to list their research areas,

available honors/directed studies positions, and student capacity. Professors will eventually have distinct logins to update their own pages.

2.2. Dynamic Pages (Community Forums)

- **Discussion Forums:** A Reddit-style interface for students to ask questions, post queries about courses or research, and interact through replies.
    - This will allow students to contribute more resources that may have been missed in the static pages.
- **Student-Posted Resources:** A centralized area where students can upload, share, and discuss resources (e.g., worksheets, test prep).
    - Not necessarily separate from the forums. A resource will be a type of post that can only be replied to and not posted as a reply to other posts.

3. Technical Stack

The project leverages a robust full-stack architecture.

Component	Technologies	Rationale
Frontend	Vite, React, TypeScript, Tailwind CSS	Vite chosen over NextJS for better performance and optimization on non-Vercel hosting environments.
Backend	Java, Kotlin, Spring Boot, Gradle **NB: Files can be written in Kotlin if you prefer Kotlin to Java.**	Spring Boot provides superior performance and security compared to Python/Node, essential for handling high-traffic forum discussions and university server compatibility.
Database & Auth	PostgreSQL, Prisma, Spring Security (SSO)	Potential future integration with CWL (Campus-Wide

Component	Technologies	Rationale
		Login) authentication.

4. Team Roles & Responsibilities

- **UI/UX Designers:** Responsible for designing the visual interface and user experience.
- **Frontend Developers:** Tasked with converting UI/UX designs into functional code using Vite and React.
- **Backend Developers:** Focused on creating APIs, managing database schemas, and implementing authentication.
- **DevOps:** Responsible for CI/CD pipelines, code reviews, and maintaining open-source documentation.
- **Human Resources:** Managing cross-team collaboration, content moderation, meeting note taking and meeting coordination.

5. Development Workflow & GitHub Guidelines

To ensure code quality and system stability, the following protocols have been established:

- **Branching Strategy:** Direct contributions to the main branch are prohibited. Developers must create separate branches and open Pull Requests (PRs).
- **Code Review:** PRs cannot be merged without approval from a Senior Developer.
- **Senior Developer Status:** Developers who contribute at least 500 lines of code will be promoted to Senior Developer, granting them PR review privileges and the ability to lead sub-teams.
- **Task Management:** A ticketing system via GitHub Issues is utilized. Developers claim tickets via Discord polls.
- **Documentation:** Backend developers are required to use Swagger to document API response layouts during the design phase.

6. Project Roadmap & Deadlines

The development lifecycle is broken down into four key phases:

- **Phase 1: Core Infrastructure & Design (Target: End of June)** Focus on database

schemas, API response design (using Swagger), overall backend development and UI/UX design.

- **Phase 2: Alpha Testing & Frontend Kickoff (Target: TBA)** Frontend teams begin implementing UI designs and connecting to developed APIs.
- **Phase 3: Beta Launch (Target: August 15th)** Deployment of the beta version hosted on Vercel.
- **Phase 4: Public Release (Target: Post-August)** Pitching the finalized platform to department head (Dr. Raymond Lawrence) and migrating hosting to official UBC servers for long-term use.

7. Expected Impact

The platform aims to serve approximately 750 new students annually (potentially reaching up to 1,500 science majors). Beyond student support, the project intends to:

- Increase visibility of the CMPS department.
- Attract external club sponsors.
- Draw major tech companies (e.g., Google, Amazon) to campus for computer science and physics career fairs, leveling the playing field with engineering and management disciplines.

8. Immediate Next Steps

1. Review the provided GitHub README for project structure and setup commands.
2. Claim open issues labeled "good first issue" on the GitHub repository.
3. Backend developers: Begin documenting API requirements and expected returns using Swagger.
4. Frontend/UI teams: Begin translating requirements into visual designs. Using a web design tool such as Figma, Canva, etc...

9. Questions and Missed Points

9.1. What architecture are we using?

The project architecture follows a variation of the **Model-View-Controller (MVC)** pattern, tailored for a modern full-stack web application. While the "View" and "Controller" are physically separated between the frontend and backend, the logic follows the core MVC principles to ensure the platform is scalable and maintainable.

1. Model (Data & Logic)

In our Spring Boot backend, the **Model** represents the data structures and the business logic of the application.

- **Database Entities:** We use **PostgreSQL** to store our persistent data.
- **Data Access:** We utilize **Prisma** (and Spring-specific JPA/Hibernate patterns) to define how data is structured, retrieved, and updated.
- **Business Logic:** This layer ensures that only "hashable" keys or valid student data are processed before being sent back to the user.

2. View (The User Interface)

Since we are building a decoupled application, the **View** is handled entirely by the frontend.

- **Technologies:** Built with **Vite, React, and TypeScript**.
- **Responsibility:** It is responsible for rendering the "Static Pages" (Information Hub) and "Dynamic Pages" (Forums).
- **State Management:** It receives JSON data from the Controller and translates it into the UI components designed by the UI/UX team.

3. Controller (The Connector)

The **Controller** acts as the intermediary between the user's actions on the React frontend and the data in the Spring Boot backend.

- **REST APIs:** The backend exposes endpoints (documented via **Swagger**) that the frontend calls to perform actions.
- **Routing:** When a student requests a specific research paper or posts a forum reply, the Controller receives that request, communicates with the Model to fetch or save the data, and returns the appropriate response.
- **Security:** This layer also handles **Spring Security**, ensuring that only authorized users (potentially via SSO/CWL) can modify sensitive data or professor profiles.

Architectural Flow Summary

1. **User Action:** A student interacts with the **View** (React) to post a resource.
2. **Request:** The View sends an API request to the **Controller** (Spring Boot).
3. **Processing:** The Controller validates the request and asks the **Model** (PostgreSQL/Prisma) to save the resource.

9.2. Isn't the scope too broad?

This question was asked in response to the “dynamic” and “static” pages. The integration of both static information and dynamic forums is designed to maximize student collaboration. While the static pages provide a curated foundation, the forum allows the community to contribute emerging resources. This ensures the website functions as a truly exhaustive

resource hub for the department.

9.3. Where can I find the functional requirements and can I add some?

The current functional requirements for the sites exist as issues on GitHub. Furthermore,by utilizing the **"under review"** label, we can ensure that the project scope remains controlled while still encouraging the team to propose innovative features. This aligns with the "Senior Developer" review structure discussed in the meeting, keeping the repository organized as we move toward the Phase 1 goals in June.

10. AI and PR Policy (Not discussed)

We allow the use of AI tools, but we strongly encourage contributors to write and understand their own code. This is a student-first project, and we value quality, learning, and long-term growth far more than simply producing large amounts of code quickly.

AI can be a helpful tool for handling repetitive tasks, improving productivity, and assisting with simpler implementation details. However, the core design decisions, architecture, and critical thinking behind the project should come from the students building it. As students ourselves, we understand the needs and expectations of our users better than any AI model can.

It is also important to recognize that AI-generated code is not always secure, efficient, or maintainable. At times, it may want to replace good code simply because it was not written in that session. Code produced by AI should always be reviewed carefully. Before committing it, ask yourself:

- Is this secure?
- Will this scale well with many concurrent users?
- Is this the best solution for the problem?
- Do I fully understand what this code is doing?

To implement this we will only allow PRs with no more than **200 lines of code at a time.**